Binary Classification of Breast Cancer Using Neural Networks: A PyTorch Implementation and Analysis

Sam Reeve

April 2026

Introduction

The aim of this report is to implement and analyse a machine learning model for binary classification while also explaining the theoretical principles that underpin its behavior. Specifically we want to classify benign and malignant tumors using a feedforward neural network trained via gradient-based optimisation. As a benchmark I have also implemented a baseline regression model, allowing us to cross examine results as experiments are conducted on the network.

Dataset Description

Shape of Data

For this task we are using the Wisconsin breast cancer dataset. The data has shape (569,30) meaning it contains 569 samples, with each sample having 30 numerical features. The output is binary, 1 representing a malignant tumor and 0 being benign. These features include, for example: size, shape, color, texture.

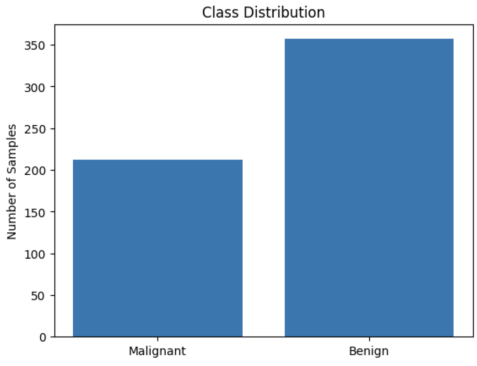

Figure 1: Class distribution plot

As we can see from the above graph, the class distribution is uneven with there being many more benign entries in the dataset. It is important to note here that this imbalance may affect the performance of our model, particularly in terms of bias towards the majority class.

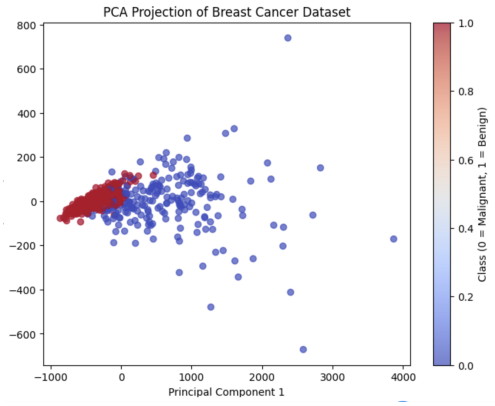

Figure 2: PCA scatter plot

Since the dataset has 30 features, visualisation in the original space is not possible. Principal component analysis (PCA) was therefore used to project the data into two dimensions while preserving as much variation as possible. The resulting scatter plot shows that the two classes are reasonably well separated. Nonetheless some overlap remains, suggesting that classification should be feasible but we should not expect our results to be perfect.

Data Split

The dataset was split into training and test sets to evaluate model generalisation on unseen data. The split I chose for this task was 80:20 for testing and training respectively. My decision to do so was guided by two facts:

- The model needs a sufficient amount of training data to reliably learn meaningful patterns
- The test set needs to be large enough to ensure performance estimates are statistically meaningful

With these facts in mind, an 80:20 split provides a suitable balance between model training and evaluation. Given that the dataset contains 569 samples, this results in 455 samples for training and 114 for testing, which is sufficient for both learning and a reliable performance assessment.

Implementing the Neural Network

To model the relationship between tumour features and their classification, I implemented a feed forward neural network. This allowed the model to develop a more complex understanding of the

data, in an attempt to find non linear patterns that our baseline regression model might fail to pick up on.

```python
#building a model

class BreastCancerModel(nn.Module):
    def __init__(self):
        super().__init__()

        #introduce the input layer with 30 features to match the shape of data
        self.layer_1 = nn.Linear(in_features=30, out_features=16)
        #add hidden reLu layer to introduce non-linearity
        self.hidden_layer = nn.ReLU()
        #output layer that has 1 out feature being either being between 0 and 1
        self.output_layer = nn.Linear(in_features=16, out_features=1)

        #define a forward method that outlines the forward pass
    def forward(self, x):
        x = self.layer_1(x)
        x = self.hidden_layer(x)
        x = self.output_layer(x)
        return x
```

Figure 3: Model architecture

The model takes 30 input features, maps them to a hidden layer of 16 neurons, applies a ReLU activation function, and produces a single output logit for binary classification. The hidden layer size was chosen as a compromise between model capacity and overfitting risk on a relatively small dataset. ReLU introduces non-linearity, while the final logit is converted to a probability using the sigmoid function. This probability is then interpretable, allowing us to classify the desired tumor.

Defining Loss Function and Optimiser

Before we get onto the training procedure, a suitable loss function and optimiser must be defined in order to measure model performance and update the model parameters.

First we will start by defining the loss function.

$$Y_i \sim \text{Bern}(p_i), \quad p_i = f_\theta(x_i) \tag{1}$$

$$f_\theta(Y_i) = f_\theta(x_i)^{Y_i} \left(1 - f_\theta(x_i)\right)^{1-Y_i} \tag{2}$$

$$L(Y_i \mid \theta) = \prod_{i=1}^{n} f_\theta(x_i)^{Y_i} \left(1 - f_\theta(x_i)\right)^{1-Y_i} \tag{3}$$

$$\ell(Y_i \mid \theta) = \sum_{i=1}^{n} \left[Y_i \log(f_\theta(x_i)) + (1 - Y_i) \log(1 - f_\theta(x_i))\right] \tag{4}$$

$$\frac{1}{n}\ell(Y_i \mid \theta) = \frac{1}{n} \sum_{i=1}^{n} \left[Y_i \log(f_\theta(x_i)) + (1 - Y_i) \log(1 - f_\theta(x_i))\right] \tag{5}$$

$$L_{BCE} = -\frac{1}{n} \sum_{i=1}^{n} \left(Y_i \cdot \log \hat{Y}_i + (1 - Y_i) \cdot \log(1 - \hat{Y}_i)\right)$$

Binary cross-entropy was used as the loss function, since maximising the Bernoulli log-likelihood is equivalent to minimising BCE (as seen clearly above). In practice, BCEWithLogits loss was used for improved numerical stability.

Now we move on to defining the optimiser. In essence an optimiser has 2 functions:

1. Minimise the loss
2. Update parameters based on gradients

The optimiser I chose to implement was Adam. Adam is a gradient based optimiser similar to SGD, however it combines momentum with adaptive learning rates to give efficient and stable convergence. A standard learning rate of 0.01 was used. Adam is a suitable choice of optimizer for this task, given the small dataset, where efficient learning over a limited number of epochs is desirable.

Training Procedure

The model was trained for 60 epochs using gradient-based optimisation. In each epoch, a forward pass was used to compute BCEWithLogits loss, after which backpropagation calculated gradients and Adam updated the parameters. After every 2 epochs, the model was evaluated on the test set to record loss and accuracy which was then used to assess generalisation.

```
epoch_count = []
loss_values = []
test_loss_values = []
accuracy = []
test_accuracy = []

torch.manual_seed(42)

#set number of epochs
epochs = 60

#training loop
for epoch in range(epochs):

  #activate training mode
  model_0.train()

  #forward pass
  y_logits = model_0(X_train).squeeze()
  y_pred = torch.round(torch.sigmoid(y_logits)) # logits -> pred probs -> pred labels

  #calculate loss/accuracy
  loss = loss_fn(y_logits, #loss function expects raw logits as input
                 y_train)
  acc = accuracy_fn(y_true=y_train,
                    y_pred=y_pred)

  #optimizer 0 grad
  optimizer.zero_grad()
```

```
  #loss backward
  loss.backward()

  #optimizer step
  optimizer.step()

  #testing
  model_0.eval()
  with torch.inference_mode():
    #forward pass
    test_logits = model_0(X_test).squeeze()
    test_pred = torch.round(torch.sigmoid(test_logits))

    #calculate test loss /acc
    test_loss = loss_fn(test_logits,
                        y_test)
    test_acc = accuracy_fn(y_true=y_test,
                           y_pred=test_pred)

  if epoch % 2 == 0:
    epoch_count.append(epoch)
    loss_values.append(loss.detach().numpy())
    test_loss_values.append(test_loss.detach().numpy())
    accuracy.append(acc)
    test_accuracy.append(test_acc)
    print(f"Epoch: {epoch} | Loss: {loss} | Test Loss: {test_loss} | Accuracy: {acc} | Test Accuracy: {test_acc}")
```

Figure 4: Training Procedure.

Experimenting

Now we have laid out the groundwork for my initial attempt at this task, I further investigated how my model could be improved through a series of experiments and adjustments. Rather than increasing the complexity of the model architecture, I instead chose to focus on modifying the training procedure so as to not increase my chance of overfitting. Two changes were investigated: weight decay and cross-validation.

The modified objective function with weight decay is given by:

$$L_{\text{reg}}(\theta) = L(\theta) + \lambda\|\theta\|^2, \quad \lambda = \text{weight decay parameter}$$

Weight decay adds a penalty proportional to the squared magnitude of the parameters, discouraging excessively large weights and improving generalisation by reducing the model's tendency to fit noise in the training data.

Cross-validation is a method used to assess model performance more reliably. Instead of splitting the data just once into training and test sets, the data is divided into k groups, called folds. The model is then trained k times. Each time, $k-1$ folds are used for training and the remaining fold is used for testing. This means every data point is used for testing once and for training the rest of the time. In theory this should improve the reliability and repeatability of our outcome. It reduces the chance of getting misleading results from one lucky or unlucky data split by averaging performance across all folds.

Results

The performance of the models was evaluated using both loss and classification accuracy on the training and test datasets. These metrics allow us to see both how the model learns patterns on the training data and then how it generalises this knowledge onto the unseen test set.

Model	**Train Loss**	**Test Loss**	**Train Accuracy** (%)	**Test Accuracy** (%)
Baseline Linear Classifier	0.3262	0.3116	92.09	94.73
Neural Network	0.0577	0.0588	98.46	98.25
Neural Network + Weight Decay	0.0533	0.0546	98.46	99.12

Table 1: Performance of the baseline linear classifier, neural network, and neural network with weight decay.

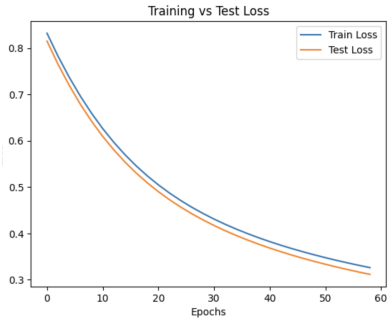

Figure 5: Baseline Regression

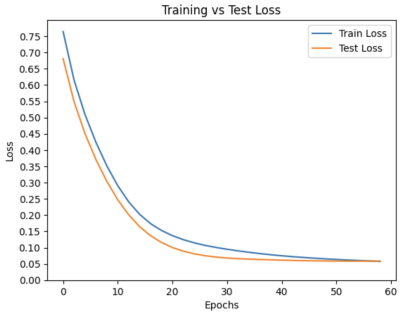

Figure 6: Neural Network

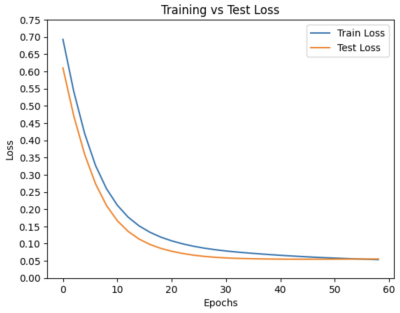

Figure 7: Neural Network + Weight decay

As seen clearly from the table, the neural networks majorly outperformed the regression model. They produced substantially lower loss and higher accuracy, suggesting the data contains patterns that are better captured by a non-linear model than by a simple linear classifier.

Additionally, we can also see that even without the introduction of a regularising term, the neural

network showed very similar training and test performance, with only a small gap between the two. This indicates that it generalised well to unseen data and showed limited evidence of overfitting.

Adding weight decay led to a small improvement in test performance, reducing test loss and increasing test accuracy. This supports the theory that discouraging large parameter values will improve generalisation, even if the effect is only marginal.

Besides this, the loss curves also support the numerical results. Because the optimiser is discouraged from exploring extreme parameter values, it makes sense that training looks more stable, and why low test loss was reached in fewer epochs.

Metric	**Mean Value**	**Standard Deviation**
Train Loss	0.0508	0.01
Test Loss	0.0820	0.03
Train Accuracy (%)	98.55	0.26
Test Accuracy (%)	97.19	1.02

Table 2: Mean and standard deviation of performance metrics for the neural network with weight decay under 5-fold cross-validation.

Although the single train-test split with weight decay produced a test accuracy of 99.12%, the cross-validation result gave a lower mean test accuracy of 97.19%. This implies that the performance in the single split may overestimate how well the model performs in general. This in turn reflects how using techniques like cross validation provide a more conservative, reliable estimate on model performance.

Additionally, the standard deviation of the test accuracy can be seen to be 1.02. This is relatively low showing the model was not incredibly sensitive to one particular partition of the dataset. As expected, mean training performance was slightly better than mean test performance under cross-validation. This reflects some overfitting, something that could be improved upon in subsequent models.

Conclusion

There is a clear pattern of improvement across the sequence of models. The baseline linear classifier performed reasonably well, but the neural network produced noticeably lower loss and higher accuracy, showing that the added flexibility of a non-linear model allowed it to capture more structure in the data. The introduction of weight decay provided further improvements to both loss and accuracy, suggesting that regularisation was successful. The implementation of cross validation, while not impoving loss or accuracy, gave useful insight to model performance by showing how results varied across different data splits, thereby improving the reliability of the overall evaluation. Broadly, the results show that each modification led to a stronger model.

In future work, the model could be improved through more systematic hyperparameter tuning. Parameters including: learning rate, weight decay strength and number of hidden layers all influence how effectively the model learns and how well it generalises. A more thorough test of how these values effect model performance could therefore lead to further improvements.